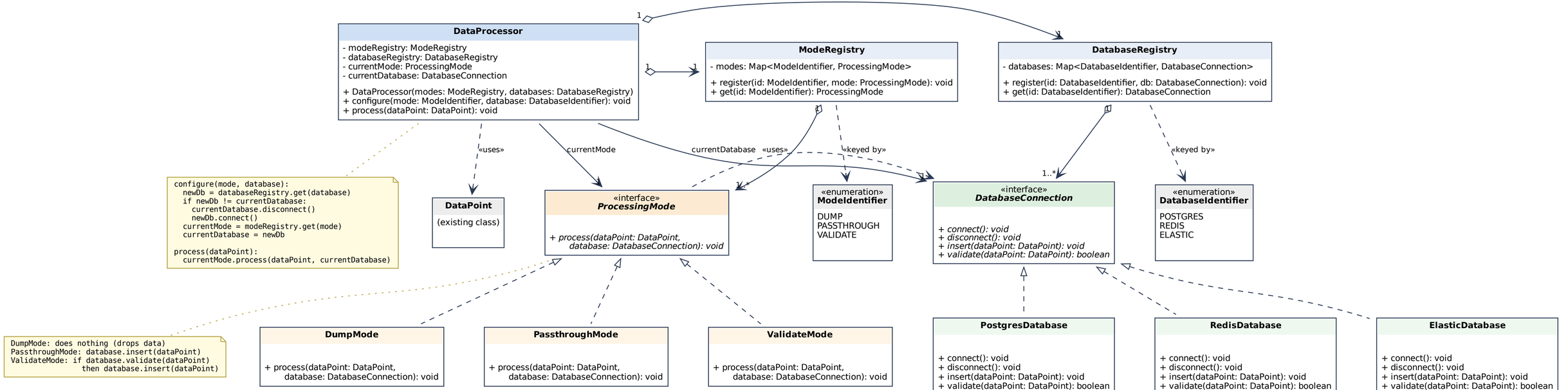
Walmart Data Processing Pipeline: Reconfigurable Data Processor

UML Class Diagram | Strategy pattern for modes and databases, registries for extension, constructor injection



Design Notes: Reconfigurable Data Processor

How it works

DataProcessor holds the current mode and the current database. **configure()** looks both up in their registries, connects the new database (disconnecting the old one only when it actually changes), and swaps the references. **process()** simply delegates: `currentMode.process(dataPoint, currentDatabase)`. Because the processor never branches on mode or database, every mode works with every database: 3 modes x 3 databases = 9 behaviours from 6 small classes.

Patterns used

Strategy (twice). ProcessingMode encapsulates *what* happens to a data point; DatabaseConnection encapsulates *where* and *how* it is stored or validated. The two strategies vary independently, which avoids a combinatorial class explosion (e.g. a ValidatePostgresMode class). This is the same separation the Bridge pattern describes.
Registry / Factory. ModeRegistry and DatabaseRegistry map each enum identifier to an instance. They replace a switch statement, and they reuse database instances so connections are not re-created on every configure call.
Dependency injection. Registries are passed into the DataProcessor constructor, so tests can inject mock modes and databases.

SOLID

Principle	How the design satisfies it
Single Responsibility	The processor routes, modes decide behaviour, databases handle storage and validation, registries handle lookup.
Open/Closed	A new mode or database is one new class plus one register() call and one enum value. No existing class is edited.
Liskov Substitution	Any ProcessingMode or DatabaseConnection implementation can be swapped in without the processor noticing.
Interface Segregation	Two small, focused interfaces rather than one large processor interface.
Dependency Inversion	DataProcessor and every mode depend only on the abstractions, never on Postgres, Redis or Elastic directly.

Assumptions

ValidateMode inserts only when `validate()` returns true; invalid data is dropped (a logging or dead-letter hook could be added later as another strategy). Datapoint, ModeIdentifier and DatabaseIdentifier are the existing classes given in the brief.